

ringio

Comprehensive Technical Engineering & Research Roadmap

Phase-by-Phase Development, Verification, & Paper Drafting Roadmap | Engineering Plan

[ENGINEERING ROADMAP OVERVIEW]

This roadmap outlines the complete engineering lifecycle for ringio across 6 structured phases: from repository architecture and zero-copy memory alignment to kernel-bypass submission ring integration, benchmarking harnesses, and publication drafting.

1. Roadmap Executive Summary

Phase	Focus Area	Key Deliverables
Phase 1	Environment & Memory Alignment	C++20 CMake build setup, liburing linking, 128-byte cache-line alignment wrappers, Google Test & Google Benchmark scaffolding.
Phase 2	Zero-Copy Page Buffer Pool	Page-aligned DMA buffer pool allocation (<code>`posix_memalign`</code>), memory page locking (<code>`mlock`</code>), pre-registered buffer mapping (<code>`io_uring_register_buffers`</code>).
Phase 3	Kernel-Bypass Submission Engine	User-space lock-free SPSC submission queue, SQPOLL kernel thread initialization (<code>`IORING_SETUP_SQPOLL`</code>), fixed file descriptor registration (<code>`io_uring_register_files`</code>).
Phase 4	Asynchronous Completion Engine	Lock-free completion queue processing, single-pass batch submission, out-of-order request correlation, zero-allocation completion event handling.
Phase 5	Microarchitectural Profiling Suite	Google Benchmark suite, IOPS scaling, p99/p99.9 tail latency measurement against POSIX pread

		baselines, ThreadSanitizer verification.
Phase 6	Paper Writing & Publication	Formulating methodology, benchmark plot generation, USENIX ATC / EuroSys / IEEE TPDS manuscript drafting.

2. Detailed Phase Breakdowns

Phase 1: Project Scaffolding & Memory Alignment Primitives

- **Repository Directory Structure Setup:** Establish clean header-only directory layout (`include/ringio/`) alongside Google Benchmark (`benchmarks/`) and Google Test (`tests/`) harnesses.
- **Build System Configuration:** Create CMake build scripts targeting C++20 standard (`-std=c++20`), enabling optimization flags (`-O3`, `-march=native`), and linking `liburing` (`-luring`).
- **Hardware Cache Alignment Wrapper:** Design strict 128-byte dual-cache-line padding structures using `alignas(128)` to isolate atomic head and tail pointers from L2 spatial prefetcher false sharing.

Phase 2: Zero-Copy Page-Aligned Buffer Pool

- **Page-Aligned Memory Allocation:** Implement a zero-allocation buffer manager using `posix_memalign()` or `aligned_alloc()` to enforce strict 4096-byte memory boundary alignment matching physical NVMe drive sectors.
- **Memory Page Locking:** Lock memory buffer pools into RAM using `mlock()` to prevent kernel page-swapping stalls during high-concurrency storage passes.
- **Kernel Buffer Registration:** Register the memory pool permanently with the kernel using `io_uring_register_buffers()` to establish fixed DMA mappings and eliminate per-request page pinning.

Phase 3: Kernel-Bypass Submission Engine & SQPOLL

- **Lock-Free Submission Ring:** Build a zero-allocation single-producer single-consumer (SPSC) and MPMC submission queue interfacing directly with `io_uring_sqe` descriptor slots.
- **Kernel Thread Polling Mode:** Initialize `io_uring` instances with `IORING_SETUP_SQPOLL` to spawn a kernel polling thread, enabling zero-syscall storage command submission.
- **Fixed Descriptor Registration:** Register NVMe file descriptors at startup using `io_uring_register_files()` to bypass kernel permission checks and file table lookups.

Phase 4: Asynchronous Completion Engine & Event Loop

- **Completion Queue Processing:** Implement non-blocking harvesting of `io_uring_cqe` completion entries from shared memory rings without executing kernel traps.

- **Out-of-Order Request Correlation:** Correlate completed I/O operations back to user requests using 64-bit opaque token IDs passed directly through completion descriptors.
- **Batched Event Dispatch:** Process completed completions in contiguous blocks (16–32 events) to minimize atomic tail updates on the completion queue.

Phase 5: Microarchitectural Profiling & Benchmarking

- **Throughput Harness:** Construct Google Benchmark programs measuring 4KB random read/write IOPS scaling across thread counts from 1 to 32 cores.
- **Tail Latency Tracking:** Record high-precision \$p50\$, \$p95\$, \$p99\$, and \$p99.9\$ latency percentiles under high queue depth saturation.
- **Baseline Comparative Analysis:** Benchmark ``ringio`` against synchronous ``POSIX pread/pwrite``, asynchronous ``libaio``, and standard ``io_uring`` without SQPOLL.
- **Correctness Verification:** Verify zero race conditions under ThreadSanitizer (``-fsanitize=thread``) and check for memory leaks under AddressSanitizer (``-fsanitize=address``).

Phase 6: Paper Composition & Publication Preparation

- **Manuscript Drafting:** Compose full systems paper detailing microarchitectural motivation, kernel-bypass design, and empirical evaluation.
- **Data Visualization:** Generate publication-ready charts showing IOPS throughput curves, tail latency percentiles, and system call count reductions.
- **Open-Source Artifact Packaging:** Package the ``ringio`` repository into a fully reproducible artifact for review at systems venues (e.g., USENIX ATC, EuroSys, or IEEE TPDS).